

AI 多模型智慧行政協作系統

文件 01 | API Specification & Model Orchestration

版本：v1.0

用途：工程開發、API 串接、資料庫設計、UI/UX 實作與驗收。

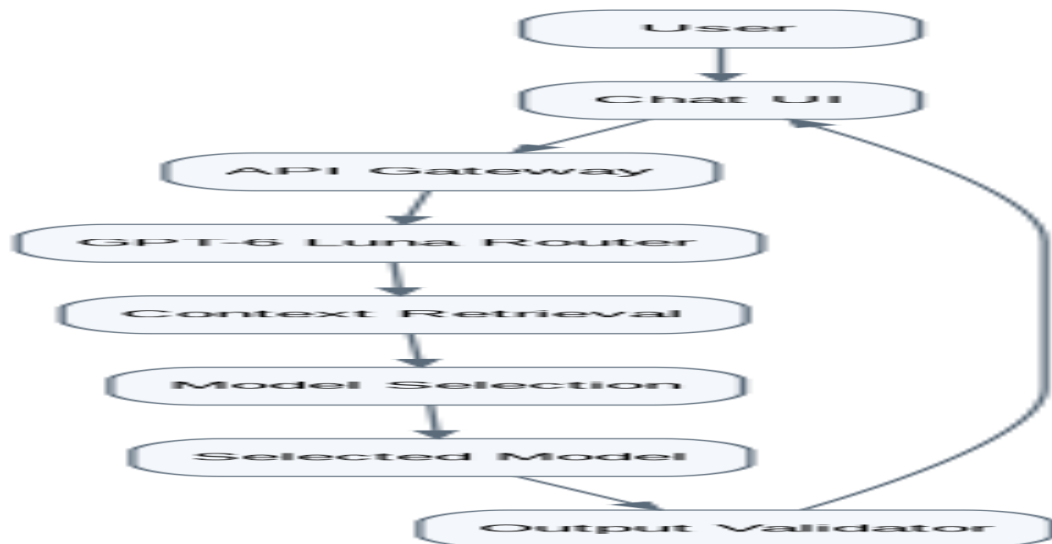
1. 文件定位

定義前端、API Gateway、GPT-6 Luna Router、Context Retrieval、五個模型 Adapter、Output Validator 之間的 API contract。模型 Provider 必須可替換，Router 與 UI 必須解耦。

模型代號：MODEL_A=Claude Sonnet 5.5、MODEL_B=Claude Opus 5.5、MODEL_C=GPT-6.1 Sol、MODEL_D=Gemini 3.8 Flash、MODEL_E=GPT-6 Luna。

2. 端到端 API Flow

圖 1：訊息處理與模型執行流程



使用者訊息先進入 API Gateway，再由 Router 判斷任務；Context Retrieval 僅取得目前 Project/Conversation 相關資料；Selected Model 執行後由 Output Validator 驗證。

3. Public API

POST /api/projects

建立 Project。Request：name、description；Response：project_id、name、created_at。

POST /api/projects/:projectId/files

上傳檔案並回傳 file_id、processing_status。原始檔進 Object Storage。

POST /api/conversations

建立 Conversation。

POST /api/conversations/:conversationId/messages

送出使用者訊息；支援 attachments、streaming。

```
POST /api/conversations/{conversationId}/messages
{
  "content": "■■■■■■■■■■",
  "attachments": ["file_uuid_1", "file_uuid_2"]
}

200
{
  "message_id": "uuid",
  "execution_id": "uuid",
  "model": "MODEL_B",
```

```

"stream": true
}

```

4. Internal Router API

POST /internal/ai/route

```

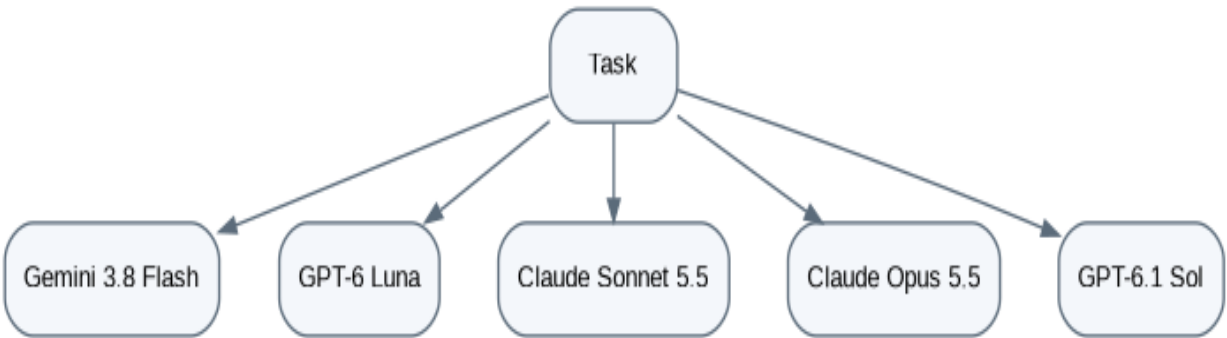
{
  "message_id": "uuid",
  "project_id": "uuid",
  "conversation_id": "uuid",
  "original_message": "...",
  "conversation_context": [],
  "project_context_summary": "",
  "available_files": []
}

{
  "original_message": "...",
  "routing": {
    "recommended_model": "MODEL_B",
    "confidence": 0.94,
    "task_complexity": "HIGH",
    "task_type": ["DOCUMENT_COMPARISON"],
    "reasoning_required": true,
    "multimodal_required": false,
    "long_context_required": true,
    "project_context_required": true
  },
  "task": {
    "objective": "...",
    "role": "...",
    "context": [],
    "source_data": [],
    "constraints": [],
    "instructions": [],
    "verification": []
  },
  "output_specification": {
    "language": "zh-TW",
    "format": "markdown",
    "required_sections": ["comparison", "recommendation"],
    "length": "medium",
    "audience": "general_administrative_user"
  }
}

```

5. Model Routing

圖 2：模型選用總覽



模型	主要任務	優先條件
MODEL_E	routine / 高量低複雜	LOW + 重複 + 明確格式
MODEL_A	一般行政 / 文件	LOW~MEDIUM + 文字工作
MODEL_B	複雜 / 長上下文	HIGH~VERY_HIGH + 多階段分析

模型	主要任務	優先條件
MODEL_C	邏輯 / 比較 / 決策	HIGH reasoning + 找錯/決策/流程
MODEL_D	多模態 / 快速抽取	PDF、圖片、圖表、表格為主

6. Model Adapter Contract

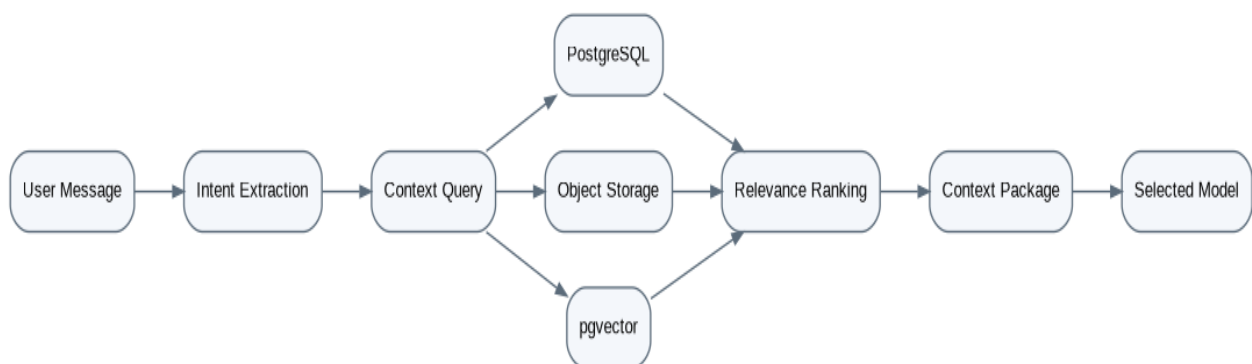
```
interface AIModelAdapter {
  execute(request: ModelRequest): Promise<ModelResponse>;
  estimateCost(request: ModelRequest): Promise<CostEstimate>;
  healthCheck(): Promise<boolean>;
}

interface ModelRequest {
  model: string;
  systemInstruction: string;
  taskInstruction: string;
  context: ContextPackage;
  outputSchema: JSONSchema;
  maxOutputTokens?: number;
}
```

Business Logic 不得直接呼叫 Provider SDK；所有模型透過 Model Registry / Adapter Layer。

7. Context Retrieval API

圖 3：Context Retrieval Pipeline



```
{
  "project_summary": "...",
  "relevant_conversation": [],
  "relevant_documents": [],
  "relevant_file_chunks": [],
  "previous_decisions": [],
  "retrieval_metadata": {
    "query": "...",
    "sources": [],
    "scores": []
  }
}
```

8. Output Validator / Error

Validator 檢查 JSON schema、required sections、語言、內容完整性、Project scope、敏感資料暴露與不當確定性。最多 retry 2 次；仍失敗時依任務類型切換 fallback model。

```

PROVIDER_TIMEOUT
PROVIDER_RATE_LIMIT
PROVIDER_AUTH_ERROR
PROVIDER_INVALID_REQUEST
PROVIDER_SERVER_ERROR
MODEL_OUTPUT_INVALID
CONTEXT_TOO_LARGE
UNKNOWN_ERROR

```

9. Observability / SLA

指標	目標
Router decision	< 3 秒
Chat TTFT	< 5 秒
API acknowledgement	< 500 ms (不含模型生成)
Retry	最多 2 次
Execution log	model / tokens / latency / TTFT / cost / status

附件：圖表名稱 + 原 UML / Mermaid

圖 1：訊息處理與模型執行流程

正文第 2 節原 UML / Mermaid：

```
graph TD
    User[User] --> ChatUI[Chat UI]
    ChatUI --> API[API Gateway]
    API --> Router[GPT-6 Luna Router]
    Router --> Retrieval[Context Retrieval]
    Retrieval --> Selection[Model Selection]
    Selection --> Model[Selected Model]
    Model --> Validator[Output Validator]
    Validator --> ChatUI
```

圖 2：模型選用總覽

正文第 5 節原 UML / Mermaid：

```
graph TD
    Start([User Task]) --> M1{ }
    M1 -- Yes --> D[Gemini 3.8 Flash]
    M1 -- No --> M2{ routine }
    M2 -- Yes --> E[GPT-6 Luna]
    M2 -- No --> M3{ }
    M3 -- Yes --> A[Claude Sonnet 5.5]
    M3 -- No --> M4{ }
    M4 -- Yes --> C[GPT-6.1 Sol]
    M4 -- No --> B[Claude Opus 5.5]
```

圖 3：Context Retrieval Pipeline

正文第 7 節原 UML / Mermaid：

```
graph LR
    M[User Message] --> I[Intent Extraction]
    I --> Q[Context Query]
    Q --> DB[(PostgreSQL)]
    Q --> F[Object Storage]
    Q --> V[(pgvector)]
    DB --> R[Relevance Ranking]
    F --> R
    V --> R
    R --> P[Context Package]
    P --> S[Selected Model]
```